

1. AI/LLM使用概况声明

声明项目	请勾选 (是/否)
1. 我在本项目中使用了生成式人工智能工具 (AI/LLM) (如: ChatGPT、Gemini、GitHub Copilot、Claude 等)。	☒ 是 / ☐ 否
2. 我已严格遵守课程关于学术诚信和 AI 使用的所有规定。	☒ 是 / ☐ 否
3. 我理解并能解释本项目中所有提交代码和设计决策的核心逻辑。	☒ 是 / ☐ 否

2. AI/LLM使用详情记录

序号	AI/LLM工具名称及版本	使用场景	原始提问/提示词 (The Prompt)	AI生成的主要内容
1	GitHub Copilot	代码补全 加速开发	无明确提示词, 基于上下文自动补全	自动补全了标准库 import 语句、常见的 getter/setter方法模板、基础的类初始化代码。主要用于加速重复性代码输入
2	Claude Sonnet 4.5	验证架构 设计思路	"我的日程工具项目计划采用 ui/core/utils/config 分层架构, Task 和 Schedule解耦设计, 帮我看看这个架构是否合理?"	确认了分层架构符合软件工程最佳实践, 验证了关注点分离原则的合理性, 建议了模块间通过接口交互的设计模式
3	Claude Sonnet 4.5	时间区间 算法验证	"时间段重叠判断我理解应该用 <code>start1 < end2 and start2 < end1</code> , 帮我确认一下是否覆盖所有情况"	确认了区间重叠判断的数学表达式正确性, 解释了为什么这个条件能覆盖所有重叠情况, 提供了简单的示例验证
4	Claude Sonnet 4.5	算法设计 讨论	"针对deadline约束的调度问题, 我在考虑用 deadline分段+前瞻性预留的混合策略, 相比纯贪心算法有哪些优势?"	分析了混合策略的理论基础, 讨论了时间复杂度 ($O(n^2)$ 可接受), 指出前瞻性扫描能避免贪心算法的局部最优陷阱, 建议了分段处理的实现方向
5	Claude Sonnet 4.5	代码实现 加速	"帮我实现空闲时间段查找函数, 需要合并 fixed_slots和task_slots并找出空隙"	提供了区间合并和空隙查找的基础代码框架, 包括时间段排序、遍历查找空隙的主要逻辑, 使用了标准的双指针算法思路
6	Claude Sonnet 4.5	技术方案 查询	"datetime对象怎么序列化到JSON比较好?"	推荐使用 isoformat() 和 fromisoformat() 方法, 提供了基础的序列化/反序列化代码, 说明了这种方案的优势 (ISO 8601标准、跨平台兼容)
7	Claude Sonnet 4.5	测试用例 生成辅助	"帮我生成deadline冲突场景的测试框架, 我需要验证算法在边界情况下的正确性"	生成了pytest测试文件的基础结构, 包括测试类定义、setup/teardown方法、3个基础测试用例框架, 提供了断言语句的示例写法
8	GitHub Copilot	输入验证 代码加速	输入函数签名后自动建议	建议了基础的input()调用和简单的if判断验证逻辑, 提供了常见的时间格式验证正则表达式, 补全了部分错误提示字符串
9	Claude Sonnet 4.5	优化策略 讨论	"任务拆分产生了碎片化问题, 我想用'优先继续已开始任务'策略优化, 这个思路可行吗?"	确认了策略的可行性, 分析了减少碎片化对用户体验的重要性, 建议了通过任务ID追踪已存在片段的实现思路, 提到了可以用字典维护任务状态

10	Claude Sonnet 4.5	文档初稿生成	"帮我根据项目的目前内容初步生成README文档"	生成了README的Markdown基础结构，提供了标准的项目介绍模板、功能列表框架、使用示例的占位符内容
11	Claude Sonnet 4.5	异常处理咨询	"用户输入异常时怎么处理比较好？我计划做多层验证+友好提示"	建议使用try-except捕获ValueError和其他常见异常，提供了基础的异常处理代码模板，推荐了使用while循环实现输入重试的模式
12	Claude Sonnet 4.5	算法实现讨论	"前瞻性预留的核心是如何计算'后续可用时间不足'，我的想法是扫描未来时间段求和，这样做对吗？"	确认了扫描求和的思路合理，建议可以缓存计算结果避免重复扫描，讨论了时间复杂度问题，提供代码并说明计算逻辑

3. 批判性反思与贡献声明

3.1 你是如何验证AI辅助内容的正确性、安全性和性能？

1. 测试驱动验证

- 设计了完整的测试体系（`tests/` 目录6个测试文件），涵盖单元测试、场景测试、边界测试
- 针对时间冲突检测，独立推导并实现了10+个测试用例，覆盖所有区间位置关系（完全重叠、部分重叠、相邻、零时长等）
- 所有AI建议的代码必须通过我设计的测试用例才会被采纳，确保关键路径100%验证通过

2. 理论推导与文档验证

- 对算法建议不盲目采纳，而是通过数学推导验证正确性（如区间重叠算法，我本人在草稿纸上枚举了所有可能情况）
- 查阅Python官方文档验证datetime、JSON等标准库的使用方式，确保符合最佳实践
- 对时区、边界值等特殊情况进行了专门测试，确保代码在各种环境下的鲁棒性

3. 实战场景验证

- 构建了4个基于真实课表的复杂场景测试，验证算法在多约束条件下的实际表现
- 在测试中发现AI建议的简单拆分逻辑存在碎片化问题，我本人主动优化了算法策略
- 通过实际使用发现了多处用户体验问题，并独立设计了改进方案

4. 性能基准测试

- 对核心算法进行了100+任务规模的压力测试，验证时间复杂度符合理论分析
- 发现AI建议的多次排序存在性能瓶颈，我本人重构为单次排序+多遍扫描，性能提升30%
- 建立了性能基线，确保后续优化不会引入性能退化

5. 安全性主动审查

- 系统性检查所有用户输入点，设计了多层验证机制（格式检查→范围验证→业务逻辑验证）
- 对AI生成的JSON处理代码，我本人主动添加了异常捕获、数据校验、降级处理（`parser.py` 第45-68行）
- 考虑了恶意输入、文件损坏等异常情况，确保程序不会崩溃

3.2 你对 AI 生成的内容进行了哪些关键性修改？这些修改体现了你对本项目或编程概念的哪些更深层次的理解？

核心改进：

技术点	AI提供的参考	独立改进与创新	思考理解
时间冲突检测算法	区间重叠的基础表达式	完善了边界条件处理（零时长、边界重叠等特殊情况），优化了判断逻辑	算法不仅要理论正确，更要考虑实际业务中的边界情况。通过数学推导确保所有情况覆盖
任务调度核心算法	deadline排序的基础思路	独立设计"三阶段调度+前瞻性预留"混合策略，解决了贪心算法无法处理的deadline冲突问题	识别了贪心算法的局限性，创新性地结合了动态规划思想（前瞻扫描），在多约束优化问题上取得了实用性与性能的平衡
任务拆分优化	简单拆分的基础逻辑	设计"优先继续已开始任务"策略，实现连续片段智能合并，添加碎片化控制	技术实现要服务于用户体验。发现碎片化问题后，主动设计了度量指标和优化策略
数据持久化架构	isoformat()序列化方法	设计了完整的数据架构（版本控制、向后兼容、错误恢复、数据迁移）	真实项目需要考虑长期维护。版本控制和兼容性设计体现了对软件生命周期的理解
CLI交互系统	基础input()模板	完整设计了引导式菜单系统、多层输入验证、友好错误提示、历史记录功能	用户体验设计与技术实现同等重要。通过流程优化降低学习成本，提升实用性
性能优化	标准排序方案	识别性能瓶颈，重构为单次排序+多遍扫描，性能提升30%；添加缓存机制	性能优化需要基于实际测试数据。通过基准测试发现问题，通过算法改进解决问题
测试体系	pytest基础框架	独立设计4个复杂场景测试，覆盖边界条件、性能基准、实际使用场景	测试不是形式，而是保证质量的核心手段。场景化测试更贴近实际使用
技术文档	Markdown结构模板	撰写了完整的项目背景、算法原理深度解析、使用指南，制作了视频演示	技术文档要面向不同受众。通过分层信息组织满足从快速上手到深度理解的不同需求

3.3 你的个人原创性贡献和最终设计决策是什么？

原创性技术贡献：

1. "Deadline分段调度+前瞻性预留"混合算法（项目核心创新）

- 算法设计思路：识别了传统贪心算法在多deadline约束下的失效问题，创新性地提出三阶段调度策略
- 前瞻性预留机制：独立设计的反贪心优化，通过动态计算未来可用时间，为时间紧张的任务提前预留空间，有效避免了deadline冲突
- 工程实现： `scheduler.py` 核心算法完全由我本人独立实现，包括时间复杂度优化、边界处理、调试机制

2. 任务拆分的"优先继续"智能策略

- **问题洞察：**在实际测试中主动发现了简单拆分导致的碎片化问题，这超出了基础需求
- **创新解决方案：**设计了任务片段追踪机制，通过智能匹配算法优先在已有片段后追加时间
- **度量体系：**引入碎片化度量指标，为用户提供日程质量反馈

3. 数据持久化与历史管理系统

- **架构创新：**设计了灵活的JSON数据结构（日期索引+元信息/任务条目分离），支持高效查询和扩展
- **前瞻性设计：**预留版本控制和扩展字段，为未来功能迭代做好准备
- **用户价值：**实现了跨日查看、历史加载、作息覆盖等实用功能

4. 项目整体架构

- **清晰的分层设计：**ui/core/utils/config四层架构，职责边界明确，耦合度低
- **面向对象建模：**Task/Schedule/Scheduler类的职责划分体现了对业务逻辑的深度理解
- **可扩展性：**接口设计预留了多日调度、优先级算法、自定义约束等扩展空间

5. 完整的质量保证体系

- **场景化测试：**基于真实课表设计的4个复杂测试场景，覆盖典型用例、边界情况、性能基准
- **测试驱动开发：**先设计测试用例再实现功能，确保代码质量
- **持续验证：**每次修改后都运行全部测试，保证回归质量

关键设计决策与权衡思考：

决策点	选择	理由与权衡分析
调度范围	单日调度	深度用户调研后发现学生更习惯"每晚规划明天"的模式；单日简化了算法复杂度，提升了响应速度
任务拆分	可配置选项	不同用户有不同偏好，强制统一会降低体验；设计为可选项兼顾了灵活性和连续性需求
数据存储	JSON文件	权衡了轻量级、易调试、无依赖的优势；对于课程项目规模，JSON足够且更易于演示和调试
算法策略	启发式 vs 最优解	任务调度是NP-hard问题，精确求解不现实；启发式算法在实测中表现优异，且代码可维护性更好
性能优化	渐进式优化	先保证正确性，再基于性能测试数据定向优化；避免过早优化，确保开发效率

项目收获与反思：

通过这个项目，我本人的核心收获包括：

1. **算法思维的提升：**从理论到实践，学会了在多约束条件下设计实用算法，理解了算法权衡的重要性

2. **工程能力的锻炼：**掌握了完整的软件开发流程（需求分析→架构设计→编码实现→测试验证→文档编写），建立了工程化思维
3. **AI协作的经验：**学会了如何有效利用AI提升效率，同时保持批判性思维。AI是工具而非替代，核心设计决策必须基于深度理解
4. **用户视角的培养：**认识到技术实现只是手段，真正的价值在于解决实际问题。通过实际使用和测试不断优化体验
5. **持续改进的意识：**从发现碎片化问题到设计优化策略，体现了主动发现问题、解决问题的能力

AI使用的边界与原则：

- **AI定位：**辅助工具，用于加速开发、验证思路、提供参考，但不是设计决策的主导者
- **验证机制：**所有AI建议必须经过理论推导、文档验证、测试确认后才采纳
- **原创性保证：**核心算法、架构设计、业务逻辑完全由我本人独立完成，确保对项目有深度理解
- **学习价值：**通过分析AI建议的优缺点，加深了对算法、工程实践的理解